

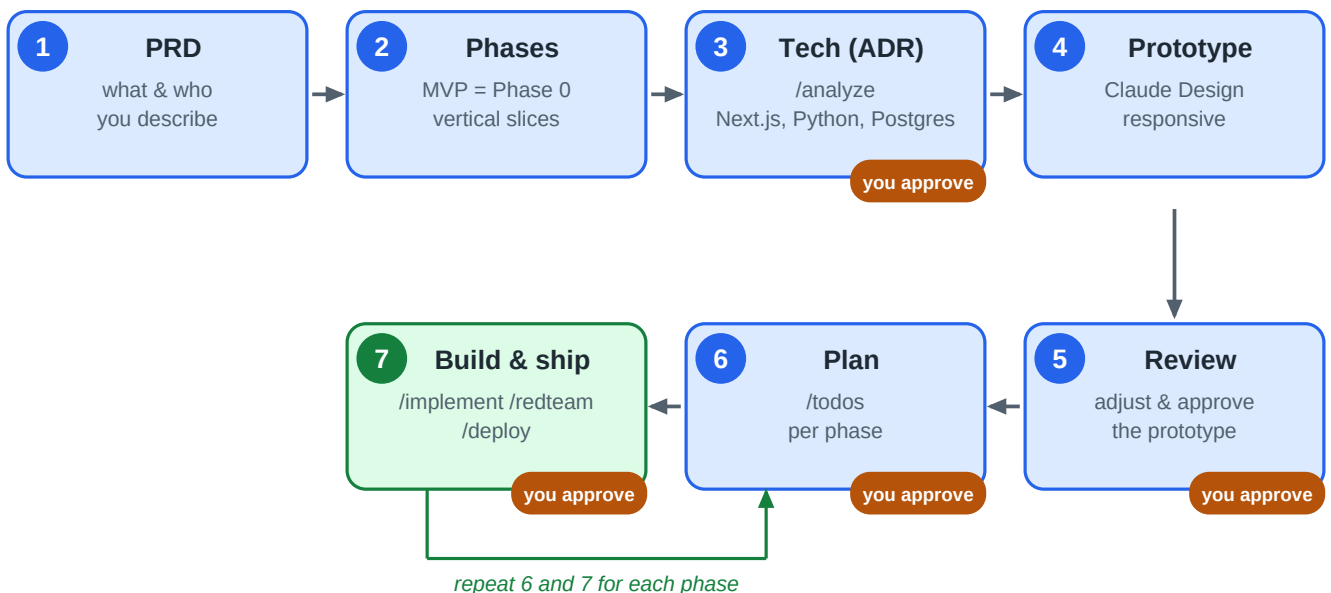
Build your first app with Claude Code

A step-by-step guide to code-starter-template for people new to vibe coding

You will not write code yourself. Your job is to **describe**, **review** and **approve**. Claude does the building, and the harness inside the template makes sure the work is planned, tested and reviewed before it reaches your users.

The example builds a web app with **Next.js** for the screens, **Python** for the logic, and **Postgres** for the data, running in **Docker Desktop** on your computer. That covers most applications except phone apps sold through an app store.

The journey at a glance



Seven steps. The orange tags mark where you approve before Claude moves on.

Before you start (once)

- Install **Claude Code**, **Git**, **Node.js** (version 22 or newer), **Python** and **Docker Desktop**. Open Docker Desktop once so it is running.
- On GitHub, open github.com/wailuen/code-starter-template and click **Use this template**. Give the new repository your project's name.
- Get it onto your computer. The easiest way: open Claude Code in an empty folder and type *Clone my repository <the link to your new repository> into this folder and open it*.
- Open Claude Code **inside the project folder**: in a terminal, go to the folder and type **claude**. The Claude desktop app and the VS Code extension work too — just open the folder.
- Type **/start** for a short orientation, then **/doctor** to check your computer has what it needs.

Good to know

When Claude asks for permission. Before Claude runs a command or changes a file, it may ask you to allow it. Read what it wants to do: if it matches what you asked for, allow it. If you are unsure, choose no and ask “*Why do you need to do that?*” Saying no never breaks anything.

Words you will see

Word	Meaning
PRD	Product requirements document: what the app does, for whom, and why
ADR	Architecture decision record: a short note of a technical choice and the reason for it
MVP	Minimum viable product: the smallest version people can actually use
Vertical slice	A small feature that works end to end: screen, logic and data together
Responsive design	One layout that rearranges itself to fit a phone, tablet or computer screen
Mobile-first	Design the phone layout first, then widen it for bigger screens
Wave	A batch of planned work; one phase becomes one or more waves
Todo	One piece of work inside a wave, with a clear list of what “done” means

Good to know

You never need to type a folder or file name. Claude knows where each file belongs in this harness.

Step 1 — Write your PRD

Talk the idea through with Claude until it is clear.

Type this

I would like to create a PRD for the below:

(Describe your idea in your own words: who will use it, what problem it solves, the main things a user should be able to do, and anything it must never do.)

Claude will ask questions. Answer them, and ask Claude to explain anything you are unsure about. When the PRD reads right to you:

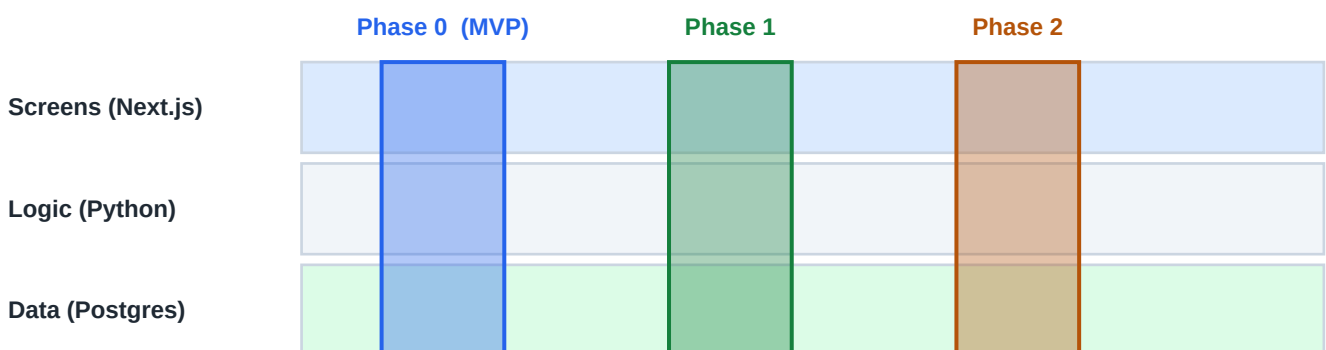
Type this

Save this PRD as the project brief.

Step 2 — Split the PRD into phases

Type this

Structure the PRD by phase, with the MVP as Phase 0, then Phase 1, Phase 2 and so on. Each phase must be a vertical slice: something a user can see and use, with the screens, backend and database working together. Keep Phase 0 small. Update the project brief.



Each phase cuts through every layer, so every phase ends with something you can click through and try.

Why

A good Phase 0 has one to three things a user can do from start to finish.

Step 3 — Decide the technology (ADR)

/analyze turns your PRD into a proper plan: it researches, writes the specifications and records the technical decisions.

Type this

`/analyze` Let's build the ADR. I would like this application to be based on a Next.js frontend, a Python backend and Postgres as the database. Everything runs locally in development, with Postgres in Docker Desktop. Use the PRD and its phases.

This step takes a while: Claude researches, writes the specifications and has the plan checked. It will:

- write the decision records (the ADR);
- fill in the project's commands (how to run tests, checks and the app) so later steps never guess;
- ask you to choose **light** mode (a one-person prototype or hobby, no real users' data, no money; fewer reviews) or **standard** mode (anything real people rely on; full reviews).

You decide

Approve the technology choices and the mode.

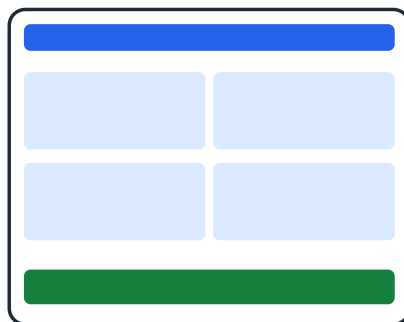
Step 4 — Design the prototype (responsive, mobile-first)

Type this

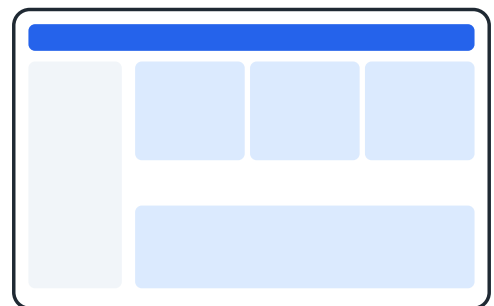
Use Claude Design to build a prototype of the whole product from the PRD and ADR, phase by phase. Make it responsive and mobile-first: show every screen at phone, tablet and desktop width. Ask me any questions about the design until you are clear.



Phone



Tablet



Desktop

Responsive design: one design that fits every screen size. Mobile-first: the phone layout is designed first.

Claude Design is included with some Claude plans. If Claude asks you to sign in to it, type **/design-login** and try again. If your plan does not include it, use this instead:

Or type this

Build the prototype as plain web pages that I can open in my browser. Make them responsive and mobile-first, and show me how to view each screen at phone, tablet and desktop width.

Step 5 — Review the prototype, then approve it

Look at every screen as if you were the user, on your phone and on your computer. Be specific about what you want changed. Example prompts:

Example 1

On the home screen the most important information is hard to find. Make it the first thing I see on a phone, and move the filters below it.

Example 2

Sign-up asks for too much. Keep only name, email and password, and ask for the rest after the first login.

Example 3

Show me every screen when there is no data yet, while it is loading, and when something goes wrong. Write the messages in plain, friendly words.

Example 4

On a phone, the menu covers half the screen. Make it a simple menu button that opens a full-screen list, and keep the main buttons within thumb reach at the bottom.

Example 5

Walk me through Phase 0 as a brand-new user, from opening the app to finishing the main task. Point out any step where I would get stuck or confused, then fix those steps.

Repeat until you are happy. Changes are cheap now and expensive after the code is built.

Approve formally

When the prototype is right, record your approval so every later step follows exactly this design:

Type this

I approve this prototype for Phase 0 (*or: for all phases*). Record my approval with my name, today's date and these words. From now on, plan and build the screens to match this approved prototype; if anything needs to differ, ask me first.

You decide

Approve the prototype, in your own words.

Step 6 — Plan the work

Type this

/todos Plan Phase 0 from the PRD phasing and the approved prototype. Each todo must deliver a working vertical slice — screen, backend and database together — that matches the approved screens at phone, tablet and desktop width.

Claude shows you the plan and asks four questions:

- Does it cover everything you described?
- Is anything there that you did not ask for?
- Is anything missing that you expected?
- Does the order make sense?

You decide

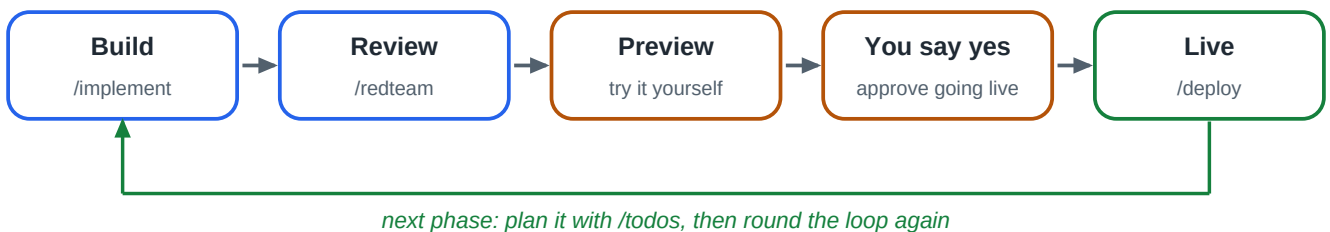
Approve the plan. Approving freezes that phase's scope. You can still change your mind later: say so, and Claude explains what changes and asks you to approve a new plan.

Step 7 — Build, review and go live

Type this

/implement Start with the first todo of the approved plan.

Make sure Docker Desktop is running first.



Every phase goes round this loop. Only you start a deploy.

- **Build** — each todo is built test-first; in standard mode an independent reviewer also checks each todo. When one todo is done, type **/implement** again for the next.
- **Review** — when every todo of the phase is built, Claude runs **/redteam** (or type it yourself). It reviews everything together, security included, then merges it.
- **Preview** — Claude shows you what changed. Try it yourself, on your phone too.
- **Go live** — the first time, type **/deploy --onboard**: Claude recommends where to host the app, tells you what it costs, and sets up alerts if the site goes down. After that, **/deploy** puts each new version live.

Then go back to step 6 for Phase 1, and repeat for each phase.

Saving your work (commit and push)

Two words you will hear a lot:

- **Commit** — save a snapshot of the project on your computer, with a short note of what changed.
- **Push** — copy those snapshots to GitHub.

Claude commits as it works. Still, ask for it whenever you finish something or stop for the day:

Type this

Commit and push my work.

Why

A safety net: if something goes wrong later, Claude can bring back any earlier snapshot. **A backup:** your work is safe on GitHub even if your computer is lost. **Sharing:** anyone you invite can see the latest version. Unfinished work stays on a separate copy (a *branch*) until it is reviewed, so pushing never puts unreviewed work live.

Along the way

Situation	What to type
Something is broken	<code>/fix</code> — if the live site is down, it first asks whether to undo the last update
Where am I? What is waiting for me?	<code>/ws</code>
Ending a working session	<code>/wrapup</code> , then <i>Commit and push my work</i> — the next session picks up from the notes
You want to change direction	Say so in plain words; Claude explains the impact and asks before changing the plan
Starting a new session	Open Claude Code in the project folder; Claude reads where you left off. To reopen your last conversation, type <code>claude --continue</code>

Tips for beginners

- **Just tell Claude what you want, in your own words.** You do not need technical terms; Claude asks when something is unclear.
- **Do not be afraid to make mistakes.** Every step is saved and reviewed, so a wrong turn can always be undone. Say *“That’s not what I meant — undo it”* and try again.
- **Be specific.** “Make the button bigger and green, at the bottom of the screen” beats “make it nicer”.
- **Ask “why?”** Claude must explain its choices in plain words. If you do not understand, say so.
- **Try it yourself** at every preview, on your phone as well as your computer.
- **Keep phases small.** A small working app beats a big unfinished one.
- **Never paste passwords or secret keys into the chat.** Claude will tell you where they go safely.